1. INTRODUCTION ABOUT THE PROJECT

In today's dynamic lifestyle and evolving real estate market, the concept of ownership is rapidly being replaced by the flexibility of renting. Whether for temporary housing, corporate events, or office setups, both individuals and businesses are increasingly seeking cost-effective and flexible furnishing solutions. The furniture rental industry has emerged as a practical alternative, allowing customers to access premium furniture without the heavy upfront investment of buying. However, managing a diverse inventory of furniture items, tracking rental durations, and handling customer billing manually is a complex and error-prone task. This is where the **Furniture Rental Management System** comes in.

The **Furniture Rental Management System** allows shop owners to bridge the gap between quality furnishing and customer needs by offering a robust, desktop-based application for managing rental operations. Designed with a focus on data accuracy, security, and ease of use, the system provides a digitalized approach to inventory and transaction management. Administrators can manage a vast category of items—such as Sofas, Tables, Beds, and Office Desks—with detailed specifications, stock status, and rental rates.

The goal of this project is to create a centralized solution that automates the tedious manual processes of the rental business. By leveraging the power of **.NET technology**, the application ensures that shop owners can efficiently track which items are out on rent, calculate total rental costs based on duration, and manage customer records seamlessly.

As the demand for rental furniture grows, this system is built to meet the operational needs of modern rental agencies. Its user-friendly Windows Graphical User Interface (GUI) ensures that even non-technical staff can navigate through booking forms, stock updates, and invoice generation with minimal training.

From a technical perspective, the platform's development is rooted in robust desktop application principles, utilizing **VB.NET** as the programming language and the **.NET Framework** for the front-end logic. The back-end is powered by a **Microsoft Access Database**, creating a self-contained and secure environment for

storing business data. The project follows a modular development approach, ensuring features like stock management, billing, and returns are organized logically across multiple forms.

By combining operational efficiency with a clean interface, the **Furniture Rental Management System** seeks to modernize traditional furniture rental shops. Whether a customer needs a single chair for a day or a full bedroom set for a month, this system offers the tools and reliability needed to manage the transaction smoothly.

2. OBJECTIVE AND SCOPE OF PROJECT

1) Objectives

The primary objective of this project is to develop the **Furniture Rental Management System**, a stable and efficient desktop application that digitizes the manual record-keeping of furniture rental shops. The specific objectives are as follows:

- **To centralize inventory management:** To provide a comprehensive digital database that hosts records of all furniture items (Name, Type, Condition, Quantity), making it easier for administrators to check stock availability in real-time and prevent over-booking.
- **To automate rental calculations:** To create a logical billing engine that automatically calculates the "Total Rent" based on the furniture type, quantity, and the number of days the item is rented, reducing human calculation errors.
- **To streamline the issue and return process:** To implement a systematic workflow for issuing furniture to customers and recording its return. This includes tracking the condition of returned items to assess any damages or fines.
- **To manage customer relationships:** To provide a secure module for storing customer details (Name, Address, ID Proof), ensuring that the business has accurate records of who is holding their assets.
- **To ensure data persistence and security:** To utilize a **Microsoft Access** database to securely store transaction history and inventory data, protecting the business from data loss associated with paper records.

2) Scope

The scope of the **Furniture Rental Management System** project encompasses the full software development lifecycle of a Windows-based application, focusing on three main modules: the User Interface (Windows Forms), the Application Logic (VB.NET), and the Database (MS Access).

1. Operational Module Scope:

- **Inventory Control:** Features to Add, Update, and Delete furniture items. Administrators can categorize items (e.g., Living Room, Bedroom, Office) and set rental prices per day/month.
- **Customer Management:** A module to register new customers and view the history of existing ones.
- **Transaction Processing:** The core functionality where the admin selects a customer, selects furniture items, defines the rental period (Start Date/End Date), and generates a rental agreement or invoice.
- **Return Management:** A dedicated section to process returned items, update stock levels back to "Available," and finalize the bill.

2. Administrator/User Scope:

- **Authentication:** A secure Login Form (`Form1`) to prevent unauthorized access to the business data.
- **Dashboard:** A main menu (`Form2` or similar) that provides quick access to all major functions like "New Rental," "Check Stock," and "Reports."
- **Reporting:** The ability to view basic reports, such as "Items Currently Rented Out" or "Total Sales," to aid in business decision-making.

3. Technical Scope:

- **Desktop Interface:** Developing a responsive and intuitive GUI using **Windows Forms** within Visual Studio, ensuring the application runs smoothly on standard Windows PCs.
- **Database Connectivity:** Implementing **ADO.NET** to establish a reliable connection between the VB.NET forms and the Microsoft Access database for performing CRUD operations.
- **Error Handling:** Integrating validation to ensure correct data entry (e.g., preventing negative stock values or invalid dates).

3(A). INTRODUCTION OF VISUAL BASIC .NET (VB.NET)

1. Overview Visual Basic .NET (VB.NET) is a robust, modern, object-oriented programming language developed by Microsoft. It is implemented on the **.NET Framework**, which provides a common set of class libraries and a runtime environment (Common Language Runtime or CLR) for execution. Launched in 2002 as the successor to the classic Visual Basic 6.0, VB.NET was designed to address the complexities of modern software development while retaining the approachable syntax that made the original Visual Basic famous. It is a "managed" language, meaning the execution of code is governed by the CLR, which handles essential tasks like memory management and security enforcement.

2. Role in the Project In the **Furniture Rental Management System**, VB.NET serves as the application's "Business Logic Layer." While the user interface is what the shop staff sees, VB.NET is the engine running behind the scenes. It is responsible for:

- **Data Processing:** Converting raw inputs (e.g., "5 Chairs for 10 Days") into meaningful financial data (e.g., "Total Rent: ₹500").
- **Logic Execution:** Implementing the rules of the rental business, such as ensuring the "Return Date" is not set prior to the "Issue Date."
- **Database Communication:** Acting as the bridge that sends commands to the Microsoft Access database to fetch inventory or save a new customer's details.

3. Key Features of VB.NET:

- **Object-Oriented Programming (OOP):** VB.NET is a fully object-oriented language. It supports the four pillars of OOP:
    - *Encapsulation:* Bundling data (e.g., Furniture ID, Rate) within classes to protect it from outside interference.
    - *Inheritance:* Creating new forms based on existing templates to maintain visual consistency across the software.

    - *Polymorphism:* allowing methods to act differently based on the object context.

- **Rapid Application Development (RAD):** VB.NET is synonymous with RAD. Its readable, English-like syntax reduces the learning curve and coding time. Features like "IntelliSense" (code completion) in Visual Studio allow developers to write code for complex tasks—like calculating overdue fines—quickly and with fewer syntax errors.
- **Structured Exception Handling:** Robust error handling is critical for a business application. VB.NET utilizes `Try...Catch...Finally` blocks to gracefully handle runtime errors. If a user inputs text into a numeric field (e.g., entering "Ten" instead of "10"), the system catches the error and displays a user-friendly message instead of crashing the application.
- **Automatic Garbage Collection:** The language utilizes the .NET Garbage Collector to automatically manage memory. It detects when objects (like a closed billing form) are no longer in use and releases the memory they occupied. This prevents memory leaks, ensuring the application remains fast and stable even after running for hours on the shop computer.
- **Interoperability and Standard Libraries:** VB.NET has access to the massive **Base Class Library (BCL)** of the .NET Framework. This project utilizes libraries such as `System.Data` (for database operations), `System.Drawing` (for UI graphics), and `System.Windows.Forms` (for screen construction).

3(B). INTRODUCTION OF WINDOWS FORMS (WINFORMS)

1. Overview Windows Forms (WinForms) is a Graphical User Interface (GUI) class library within the Microsoft .NET Framework. It provides a platform for developing rich client applications for desktop, laptop, and tablet computers running the Windows operating system. Unlike web applications that run in a browser, WinForms applications run natively on the client machine, offering superior performance, responsiveness, and access to local hardware resources.

2. Role in the Project For the **Furniture Rental Management System**, WinForms acts as the "Presentation Layer." It is the visual medium through which the shop administrator interacts with the system. WinForms allows for the creation of a professional "Dashboard" style interface where users can navigate between Inventory, Customer, and Rental modules with a single click.

3. Key Features of Windows Forms:

- **Event-Driven Architecture:** WinForms applications are interactive; they wait for the user to do something. The code is written to respond to specific events. For example, when the staff clicks the **"Calculate Bill"** button, a specific block of VB.NET code executes to process the math. Other events include `Form_Load` (to populate data when a window opens) or `SelectedIndexChanged` (to update the price when a different furniture category is selected).
- **Rich Control Library (Toolbox):** WinForms provides a comprehensive toolbox of drag-and-drop controls essential for data entry:
    - **DataGridView:** Used to display rows of furniture stock or customer lists in a table format.
    - **DateTimePicker:** Ensures users select valid dates for rental issuing and returning.
    - **ComboBox:** Allows users to select predefined categories (e.g., "Sofa", "Table") to prevent spelling errors.
    - **TextBox & Label:** Fundamental controls for entering customer names and displaying calculated amounts.

- **Visual Inheritance & Consistency:** This feature allows developers to create a base form with a specific layout, color scheme, and logo. All other forms in the project can inherit this base design. This ensures that the Login Screen, Billing Screen, and Stock Screen all share a consistent branding and visual identity.
- **Data Binding:** WinForms supports complex data binding. UI elements can be bound directly to data sources (like a `DataTable` from the database). This means if the stock quantity changes in the database, the `DataGridView` on the screen can automatically update to reflect the new number without writing complex refresh logic.

3(C). INTRODUCTION OF MICROSOFT ACCESS

1. Overview Microsoft Access is a Database Management System (DBMS) from Microsoft that combines the relational Microsoft Jet Database Engine with a graphical user interface and software-development tools. It is part of the Microsoft 365 suite. While large enterprises use SQL Server, MS Access is the industry standard for small-to-medium-scale desktop applications due to its lightweight nature and ease of deployment.

2. Role in the Project MS Access serves as the "Data Persistence Layer" for the project. It acts as the digital ledger for the furniture shop, replacing physical logbooks. It stores relational data across multiple tables, ensuring that even if the computer is turned off, the record of who rented what remains safe. The file extension used is typically `.accdb` or `.mdb`.

3. Key Features of Microsoft Access:

- **Single-File Architecture:** The entire database—including tables, indexes, and relationships—is stored in a single file on the hard drive. This makes the **Furniture Rental Management System** highly portable. The shop owner can easily backup the entire business history simply by copying the database file to a USB drive or cloud storage.
- **Relational Database Management System (RDBMS):** Access allows the creation of a normalized database structure using primary and foreign keys.
    - *Example:* The `tblRentals` table is linked to the `tblCustomers` table via a `CustomerID`. This ensures "Referential Integrity"—the system will not allow a rental booking to be created for a customer who does not exist in the database.
- **ADO.NET Connectivity via OLEDB:** Access integrates seamlessly with VB.NET through the `System.Data.OleDb` namespace. The application uses SQL queries (Structured Query Language) to interact with the data:
    - `SELECT` to find available furniture.
    - `INSERT` to add a new customer.

    - `UPDATE` to decrease stock when items are rented.

- **Cost-Effective and Low Maintenance:** For a desktop application running on a standalone PC in a shop, Access eliminates the need for installing and configuring complex database servers. It requires minimal maintenance and runs efficiently with the standard hardware resources found in typical retail environments.

4. DEFINITION OF PROBLEM

The traditional method of managing a furniture rental business relies heavily on manual record-keeping, physical ledgers, and ad-hoc spreadsheet calculations. While this approach may function for very small operations, it becomes unsustainable as the inventory grows and the customer base expands. The **Furniture Rental Management System** aims to resolve several critical inefficiencies inherent in the manual process. The primary problems identified are as follows:

1. Inefficient Inventory Tracking and "Ghost" Stock One of the most significant challenges in the manual system is maintaining an accurate real-time count of available furniture. Shop owners often struggle to track which items (e.g., specific sofa sets, dining tables) are currently out on rent and which are in the warehouse. This leads to **"Ghost Inventory,"** where staff might promise an item to a new customer, only to realize later that the physical stock is depleted. Conversely, available items may sit idle because the ledger wasn't updated upon a return, leading to lost revenue opportunities.

2. Complex Billing and Revenue Leakage Calculating rental fees manually is prone to human error. The cost depends on multiple variables: the specific **Category Rate**, the **Quantity** of items, and the exact **Number of Days** rented.

- **Calculation Errors:** Staff may miscalculate the total days (especially across different months) or apply the wrong rate, resulting in financial loss or customer disputes.
- **Late Return Fines:** Manually tracking the "Promised Return Date" vs. the "Actual Return Date" is difficult. Overdue fines are often missed or miscalculated, causing revenue leakage for the business.

3. Lack of Centralized Customer Data In a manual system, customer details (names, addresses, phone numbers) and security documents (ID proofs) are stored in physical file folders.

- **Retrieval Delay:** Finding a specific customer's rental history or contact information requires sifting through piles of paperwork, wasting valuable time.

- **Data Redundancy:** The same customer's details might be written down multiple times for different transactions, leading to cluttered and inconsistent records.

4. Difficulty in Condition Assessment and Damage Disputes Furniture is an asset that depreciates and can be damaged. Manual systems lack a structured way to record the **Condition** of an item at the time of issuance (e.g., "Minor scratch on left leg"). When the item is returned, disputes often arise between the shop owner and the customer regarding new damages versus pre-existing ones. Without a digital log connected to the specific rental ID, resolving these conflicts is subjective and difficult.

5. Data Security and Preservation Risks Physical ledgers are vulnerable to wear and tear, loss, theft, or damage from environmental factors (water, fire). If a master ledger is lost, the business loses its entire record of pending collections and asset locations. Furthermore, manual books offer no **Access Control**—anyone entering the shop can potentially read sensitive business financial data or customer information.

6. Absence of Analytical Reporting A manual system provides no immediate insight into business performance. Shop owners cannot easily answer critical questions such as:

- "Which furniture category is rented the most?"
- "Who are the customers with currently overdue items?"
- "What is the total revenue generated this month?" Compiling these reports manually requires hours of tallying, whereas a computerized system can generate them in seconds.

Conclusion of Problem Definition The cumulative effect of these issues is a rental operation that is reactive rather than proactive, prone to financial discrepancies, and inefficient in its service delivery. The **Furniture Rental Management System** is designed specifically to automate these workflows, ensuring data integrity, accurate billing, and secure inventory management.

5. SYSTEM ANALYSIS

System analysis for the **Furniture Rental Management System** involves understanding the unique requirements of the furniture shop owners (Administrators) and the operational staff. The system must provide a seamless interface for booking furniture while ensuring the logical integrity of stock levels (preventing negative stock). The system analysis phase includes identifying key functionalities that will make the application efficient and reliable using **VB.NET**.

1. Problem Identification:

- Traditional rental shops rely on paper ledgers, leading to "Ghost Inventory" (thinking an item is available when it isn't).
- Manual calculation of rental days often leads to revenue leakage, especially when customers extend their rental period without notice.

2. User Requirements:

- **Operational Staff:** Need a simple form to select a customer, select multiple furniture items, and generate a bill instantly. They require a "Search" feature to find item rates quickly.
- **Administrators:** Require features for adding new furniture types to the database, setting rental rates, modifying customer details, and viewing overall stock reports.

3. System Requirements:

- **Functional:**
    - **Authentication:** A secure Login Form (`Form1`) to grant access to the application.
    - **Inventory Management:** Modules to Add, Update, and Delete furniture items in the MS Access database.
    - **Billing Engin:** A logic-driven form where the system calculates the final amount based on date selection.

    - **Return Processing:** A function to mark items as "Returned" and automatically update the "Available Quantity" in the stock table.

- **Non-Functional:**
    - **Data Persistence:** The ability to store data reliably in a local `.accdb` file without needing an internet connection.
    - **Usability:** A consistent "Windows-style" interface using standard controls (Buttons, TextBoxes, DataGrids) for ease of use.
    - **Performance:** Instant retrieval of customer and stock records using optimized **ADO.NET** queries.

4. System Architecture:

- The system utilizes a **Desktop Application Architecture** (Client-Server logic running locally).
- **Front-end (View):** Developed using **VB.NET (Windows Forms)** within the .NET Framework. It uses graphical components to interact with the user.
- **Middle-ware (Logic):** VB.NET code-behind files handle the business logic (calculations, validations, and event handling).
- **Back-end (Data):** Powered by **Microsoft Access (OLEDB)**, utilizing relational tables (`tblItems`, `tblCustomers`, `tblRentals`) to store data.

5. Data Flow:

- The user inputs data (e.g., "Rent 2 Sofas for Customer John") via the Windows Form.
- **VB.NET** scripts validate this input (checking if 2 Sofas are actually available).
- If valid, the system calculates the cost and executes an `INSERT` query to save the rental record and an `UPDATE` query to decrease the stock in the **MS Access** database.
- Reports and Grids retrieve this data using `SELECT` queries to display current business status.

6. User Interface and Experience:

- The system emphasizes functionality and clarity. Input forms are designed with logical tab orders and clear labels.
- **DataGridViews** are used extensively to show lists of items and customers, allowing for easy sorting and selection.

7. Security Considerations:

- **Input Validation:** The system is designed to prevent logical errors, such as entering a "Return Date" that is earlier than the "Issue Date."
- **Access Control:** The database file is protected, and the application requires valid credentials to launch, ensuring that financial data remains private.

6. SYSTEM REQUIREMENTS

Minimum Hardware Requirements for Development

- **CPU:** Intel Core i3 or equivalent (modern multi-core processors ensure faster compilation in Visual Studio).
- **Memory (RAM):** 8 GB (4 GB is the absolute minimum, but 8 GB is recommended for running Visual Studio smoothly).
- **Hard Disk:** 512 GB SSD (Faster storage is crucial for loading large .NET projects and database files).
- **System Type:** 64-bit Operating System (Required for modern versions of Visual Studio).

Minimum Software Requirements for Development

- **Operating System:** Windows 10 or Windows 11 (VB.NET development is native to the Windows ecosystem).
- **IDE:** Microsoft Visual Studio 2019 or later (Community, Professional, or Enterprise editions).
- **Database:** Microsoft Access 2013 or later (for creating `.accdb` files).
- **Framework:** .NET Framework 4.7.2 or later.
- **Database Engine:** Microsoft Access Database Engine (ACE.OLEDB.12.0) to allow VB.NET to communicate with Access.

Minimum Hardware and Software Requirements for Running the Application

On Client PCs (Shop Computers)

- **CPU:** Intel Pentium Gold or equivalent (sufficient for running standard Windows Forms applications).
- **Memory (RAM):** 4 GB (Adequate for the OS and the application to run simultaneously).

- **Hard Disk:** 128 GB (The application executable and Access database file are lightweight, requiring less than 500 MB).
- **Operating System:** Windows 10 or Windows 11.
- **Prerequisites:** .NET Framework Runtime (usually pre-installed on modern Windows) and Microsoft Access Runtime (if full Office is not installed).
- **Screen Resolution:** 1366x768 or higher (Standard laptop resolution is sufficient for the forms designed).

7. SYSTEM DESIGN AND CODING

INTRODUCTION:

Software design is a critical phase in the software engineering process for the **Furniture Rental Management System**, laying the foundation for all subsequent development activities. It transcends development paradigms, serving as a common thread in the creation of every engineered system. Design is initiated after the specific requirements for the furniture shop's operations have been specified and analyzed. This step forms the core of the development phase and precedes coding and testing. The primary goal of software design is to create a model of the system that will eventually be implemented as a functional desktop application.

The central focus of the design is quality. Through the design process, the quality of the **Furniture Rental Management System** is ensured by translating shop owner and operational staff requirements into a structured framework that can be coded using **VB.NET** and **Microsoft Access**. A robust design minimizes the risk of application crashes, ensures data integrity during stock updates, and guarantees that the system remains easy to maintain as the business grows.

INPUT DESIGN:

Input design is the process of converting user-oriented input into a computer-readable format. For the **Furniture Rental Management System**, input design involves capturing customer details, selecting inventory items, and defining rental periods. The collection of input data represents a critical aspect of the system, as inaccurate stock entries or customer data can lead to business losses. Effective input design focuses on optimizing the Windows Forms—such as the "Add New Item" screen or "Rent Furniture" interface—to ensure accuracy and speed.

The primary objectives of input design for the system are as follows:

1. **Efficiency:** Create an input process that allows staff to generate a rental agreement quickly, minimizing the customer's wait time.
2. **Accuracy:** Achieve the highest possible accuracy in input data (e.g., preventing the rental of items with "0" stock) to minimize inventory mismatches.
3. **User-Friendliness:** Ensure that the input forms (Textboxes, ComboBoxes, DatePickers) are intuitive and logically arranged, making it easy for non-technical staff to use the software.

Input Data Considerations

When designing input data for the **Furniture Rental Management System**, the goal is to create a process that is easy, logical, and error-free. The data entry operators (shop staff) must be clear on which fields are mandatory (e.g., Customer Phone Number) and the correct format for data (e.g., Currency). A well-designed Form provides these cues via labels and message boxes to avoid ambiguity.

The input process typically involves the following stages:

- **Data Recording:** Capturing raw data from users (e.g., typing a customer's name into `txtCustomerName`).
- **Data Validation:** Checking the accuracy of input data against predefined criteria (e.g., ensuring `dtpReturnDate` is not earlier than `dtpIssueDate`).
- **Data Conversion:** Converting user inputs (like text in a textbox) into data types that VB.NET can process (e.g., converting "500" String to Integer for calculation).
- **Data Control:** Ensuring the integrity of the data (e.g., using parameterized queries to prevent SQL Injection attacks on the Access database).
- **Data Storage:** Transmitting the validated data via ADO.NET to be stored permanently in the Microsoft Access tables.

Input types for the rental system can be classified as:

- **External Input:** Data sourced from external entities, such as customer ID proof numbers or phone numbers.
- **Internal Input:** Data generated internally, such as automatically generating a unique `RentalID` or capturing the current system date/time.
- **Operational Input:** Data required for system operations, such as the Admin username and password for logging in.
- **Interactive Input:** Input provided by users in real-time, such as selecting "Sofa Set" from a ComboBox, which immediately triggers a query to fetch the "Price Per Day" for that item.

OUTPUT DESIGN:

Output design refers to the process of determining how the results of processing will be communicated to the users. For the **Furniture Rental Management System**, outputs play a crucial role in conveying meaningful information, whether it represents a list of available furniture, a generated bill, or a list of overdue customers. Designing effective output involves organizing the presentation of data in Grids, Labels, and Message Boxes in a manner that is clear and actionable.

The outputs of the system are typically categorized as follows:

1. **External Outputs:** Information presented to the user for external use, such as a "Rental Invoice" displaying the final amount to be collected from the customer.
2. **Internal Outputs:** Information generated within the system for internal use, such as the "Current Stock" count displayed in a DataGridView.
3. **Operational Outputs:** Outputs produced as part of routine system operations, such as "Success" message boxes after saving a record or "Invalid Login" alerts.
4. **Interactive Outputs:** Outputs that respond dynamically to user queries, such as filtering the inventory list to show only "Office Furniture" when a specific category is selected.
5. **Turnaround Outputs:** Outputs used by the system for processing and returned for further action, such as displaying the "Total Fine" calculated when a user enters a late return date.

The key to designing effective outputs for the **Furniture Rental Management System** is ensuring that the right information is presented at the right time. For example, the system immediately alerts the user if they try to rent more chairs than are available in stock.

CONCLUSION:

In the software engineering process for the **Furniture Rental Management System**, system design is foundational for building a reliable and user-friendly desktop application. Input design ensures that inventory and customer data are captured accurately, while output design guarantees that shop owners receive clear financial and stock status updates. By focusing on these design principles using **VB.NET and Windows Forms**, developers can ensure that the system meets the practical needs of the rental business, operates efficiently on standard PCs, and provides a stable platform for managing daily operations.

8. DATABASE DESIGN

TABLE 1: Bill:

Field Name	Data Type
invoiceid	AutoNumber
date	Short Text
cname	Short Text
customermobile	Short Text
noofitem	Short Text
totalbill	Short Text
paymentmode	Short Text

TABLE 2: User:

Field Name	Data Type
userid	Short Text
password	Short Text
contactnumber	Short Text
emailid	Short Text

TABLE 3: Products:

Field Name	Data Type
itemname	Short Text
noofitem	Number

9. DATA FLOW DIAGRAM

It is a graphical tool used to describe and analyze the flow of data through a system. It focuses on the data flowing into the system, between processes, and in and out of data stores.

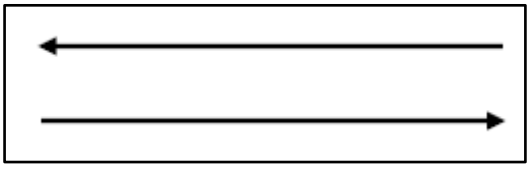
DFDs are of two types:

1. **Physical DFD:** The physical DFD is a model of the current system and is used to ensure that the current system has been clearly understood. It depicts *how* the system will be implemented (e.g., hardware, software, people).
2. **Logical DFD:** Logical DFDs are the model of the proposed system. They clearly show the requirements on which the new system should be built, focusing on *what* the system does rather than *how* it does it.

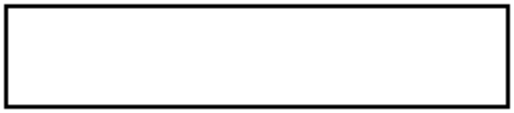
Notation Used In DFD:

There are four simple notations used to complete DFDs:

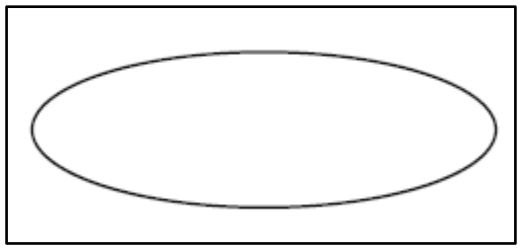
- **Dataflow:**



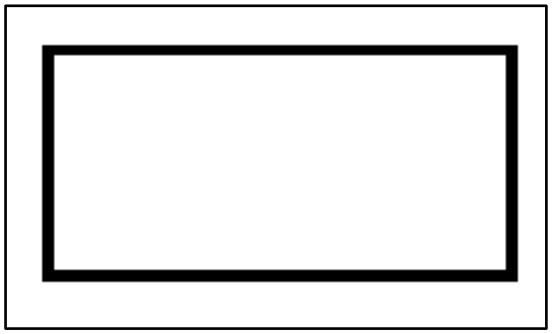
- **External Entity:**



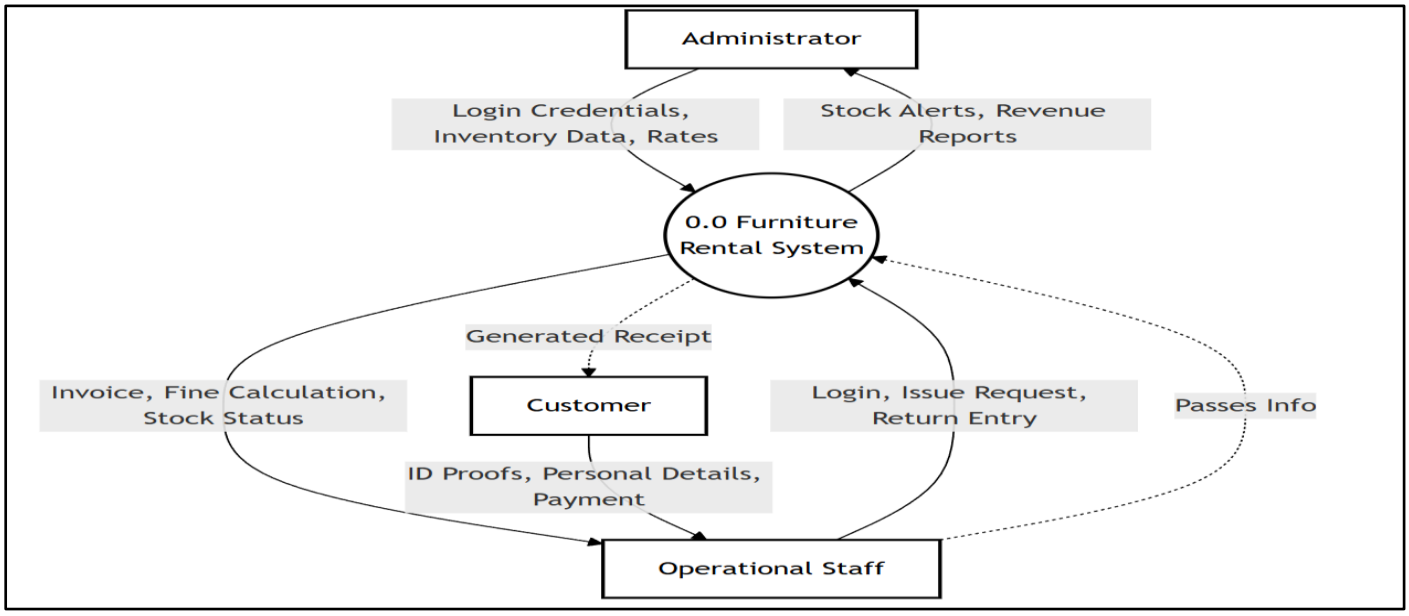
- **Process:**

- 

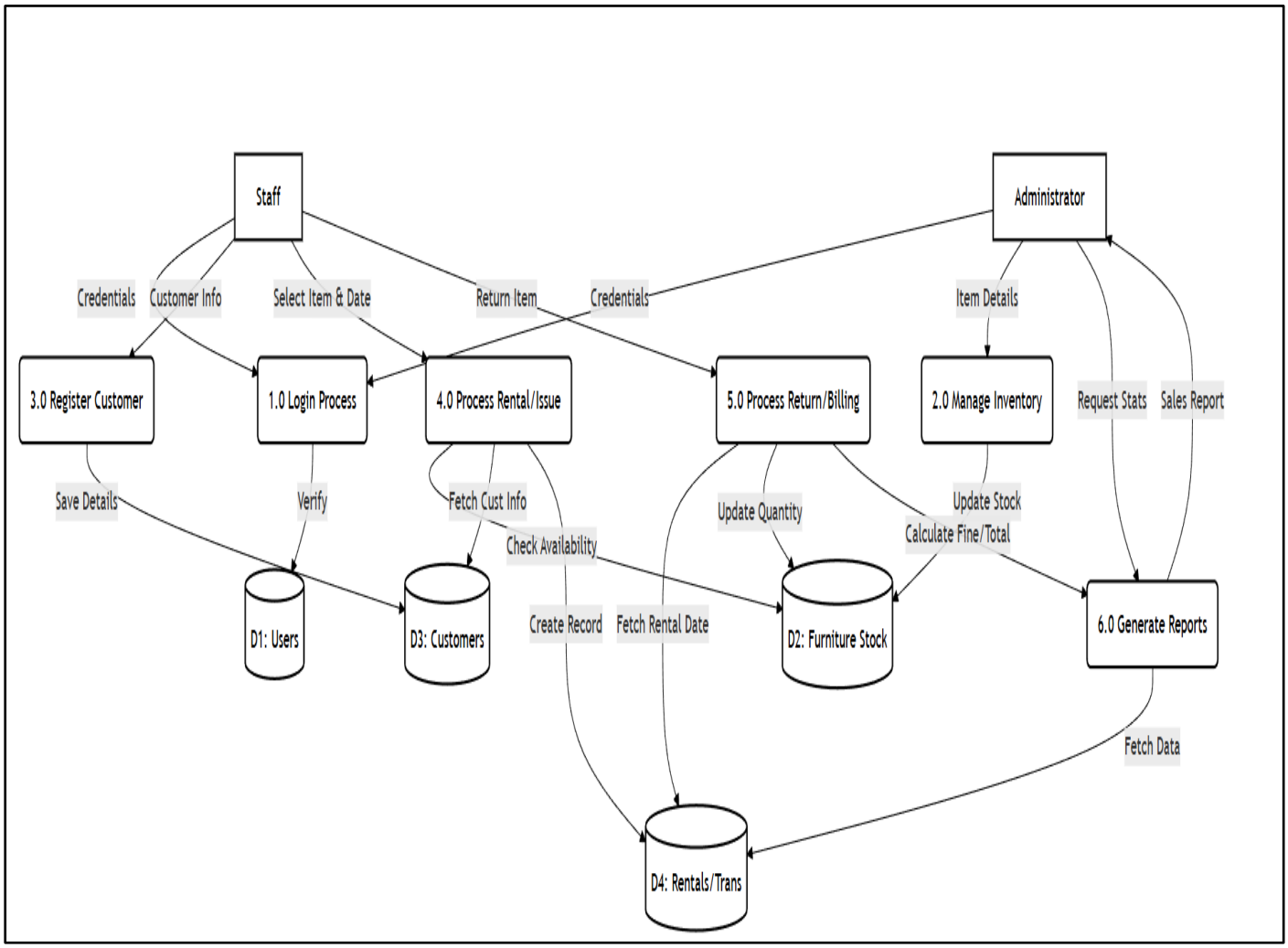
Data store:



DFD Level 0:



DFD Level 1:



10. ENTITY RELATIONSHIP DIAGRAM

An Entity Relationship Diagram (ERD) is a graphical representation that depicts the logical structure of a database. It visualizes the relationships between different entities (tables) within the **LuxAura Watch Store** database, ensuring that data is organized efficiently and without redundancy.

The core components of an E-R Diagram are:

1. Entity: An entity is a real-world object or concept that is distinguishable from all other objects. In the context of **LuxAura Watch Store**, key entities include:

- **User:** Represents the customer or admin accessing the system.
- **Product:** Represents the watches available for sale.
- **Order:** Represents a purchase made by a user.
- **Category:** Represents the classification of watches (e.g., Analog, Digital).

2. Attribute: Attributes are the specific properties or details that describe an entity. For example:

- **User Entity:** Name, Email, Password, Address.
- **Product Entity:** Watch Name, Brand, Price, Stock Quantity.

3. Relationship: A relationship is an association among several entities. For example:

- A **User** *places* an **Order**.
- An **Order** *contains* multiple **Products**.
- A **Product** *belongs to* a **Category**.

4. Weak Entity: A Weak Entity is an entity that does not have a primary key of its own. It cannot be uniquely identified by its own attributes alone and depends on the existence of another entity (known as the "Owner" or "Strong" entity).

- **Symbol:** Represented by a double-bordered rectangle.

- **Example in LuxAura Watch Store:** The **Order Details** (or Order Items) entity is a weak entity. It cannot exist without a corresponding **Order**. If an order is deleted, the specific details (like "2 units of Casio G-Shock") associated with that order are also removed.

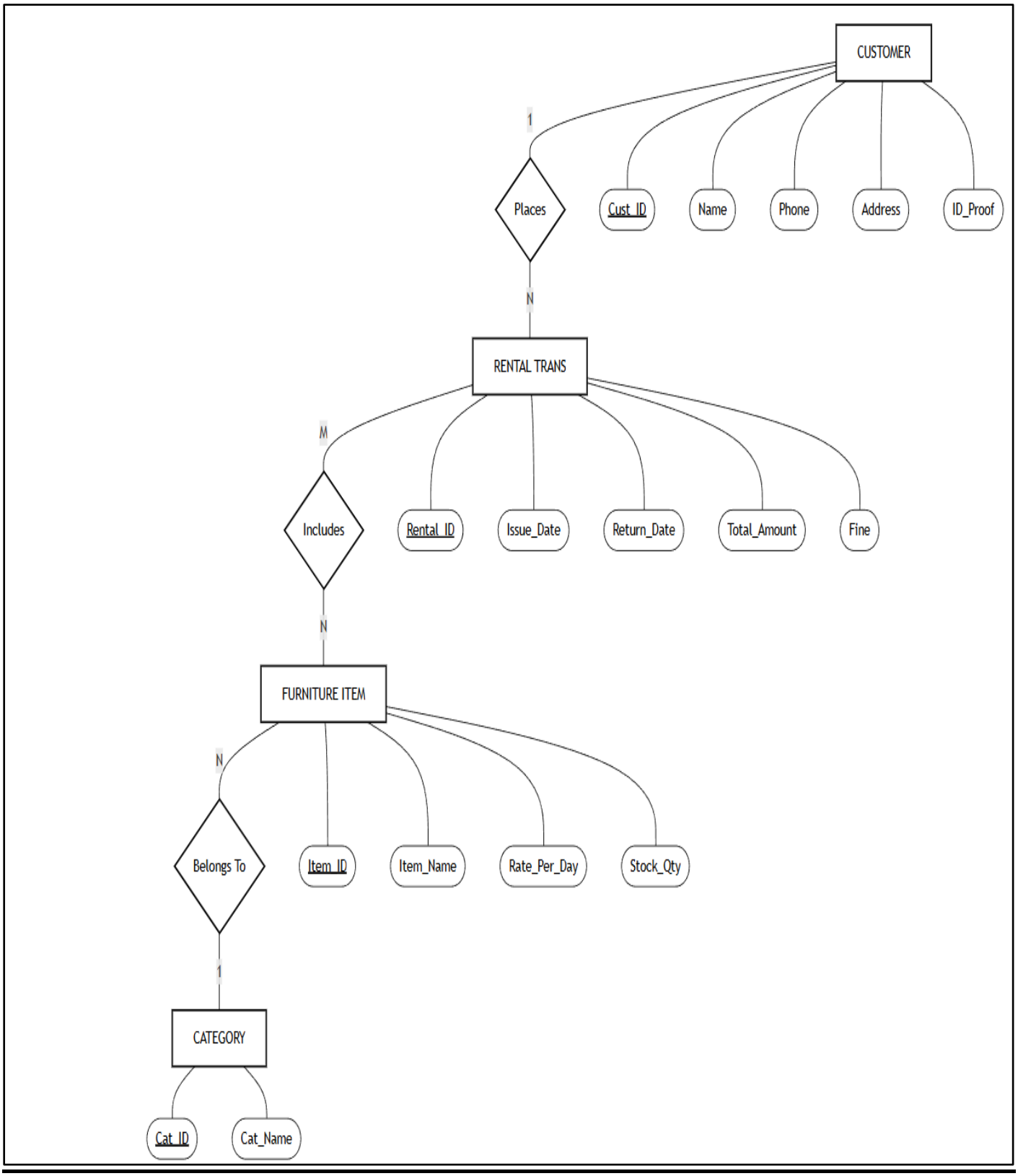
5. Attributes: An attribute is a property or characteristic of an entity. It describes the data we want to store for each entity.

- **Symbol:** Represented by an oval or ellipse.
- **Example in LuxAura Watch Store:**
    - For the **Product (Watch)** entity, attributes include: `Watch_ID`, `Brand_Name`, `Price`, `Dial_Color`, and `Strap_Material`.
    - For the **User** entity, attributes include: `User_ID`, `Email`, `Phone_Number`, and `Shipping_Address`.

Types of Attributes:

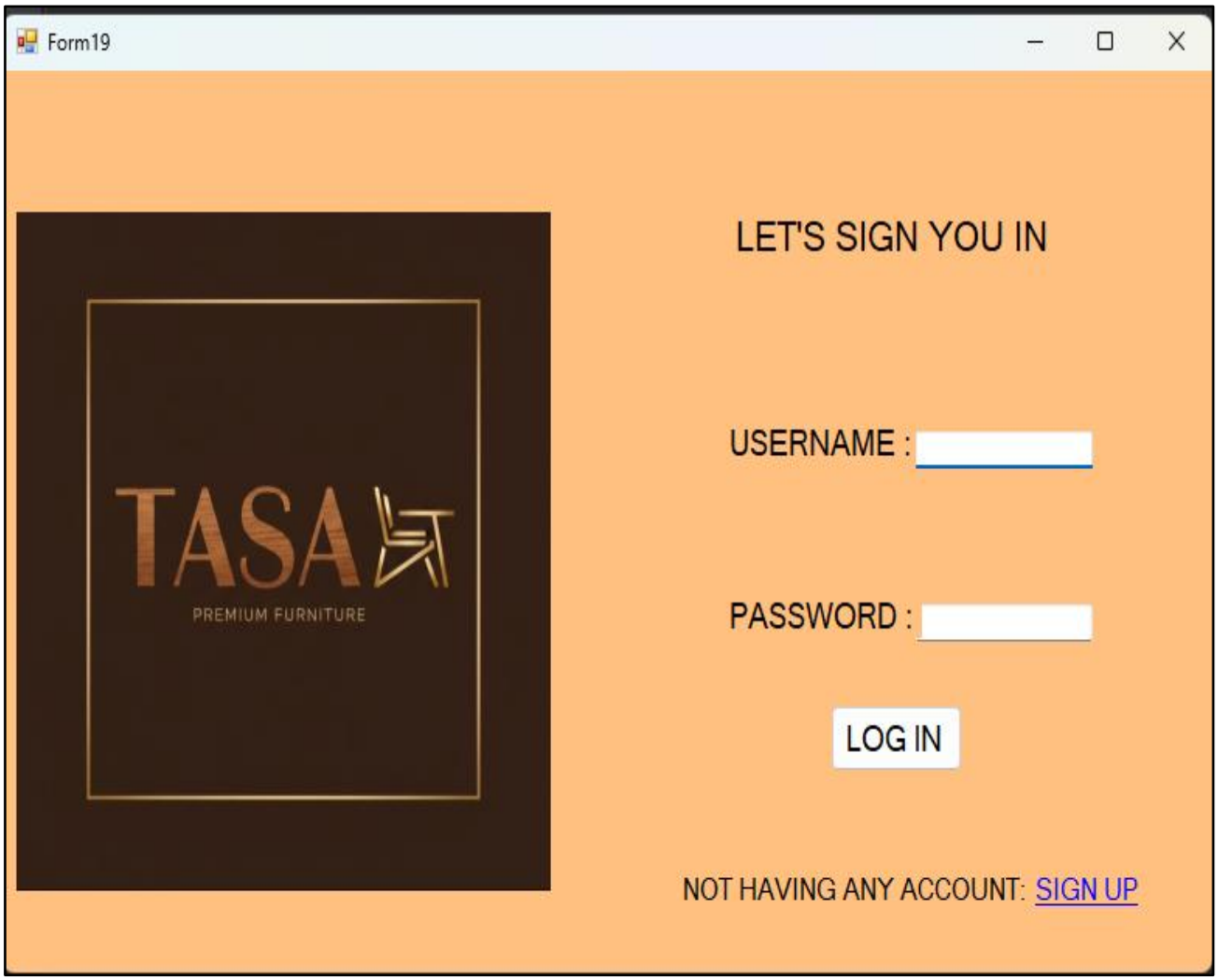
- **Key Attribute:** An attribute that uniquely identifies an entity (e.g., `Order_ID`).
- **Composite Attribute:** An attribute that can be divided into sub-parts (e.g., `Address` can be divided into `City`, `State`, `Zip Code`).
- **Derived Attribute:** An attribute whose value is derived from other attributes (e.g., `Total_Order_Value` is calculated from `Price` × `Quantity`).

ENITTY RELATIONSHIP DIAGRAM:



11. INPUT FORM AND CODING:

Login Window:

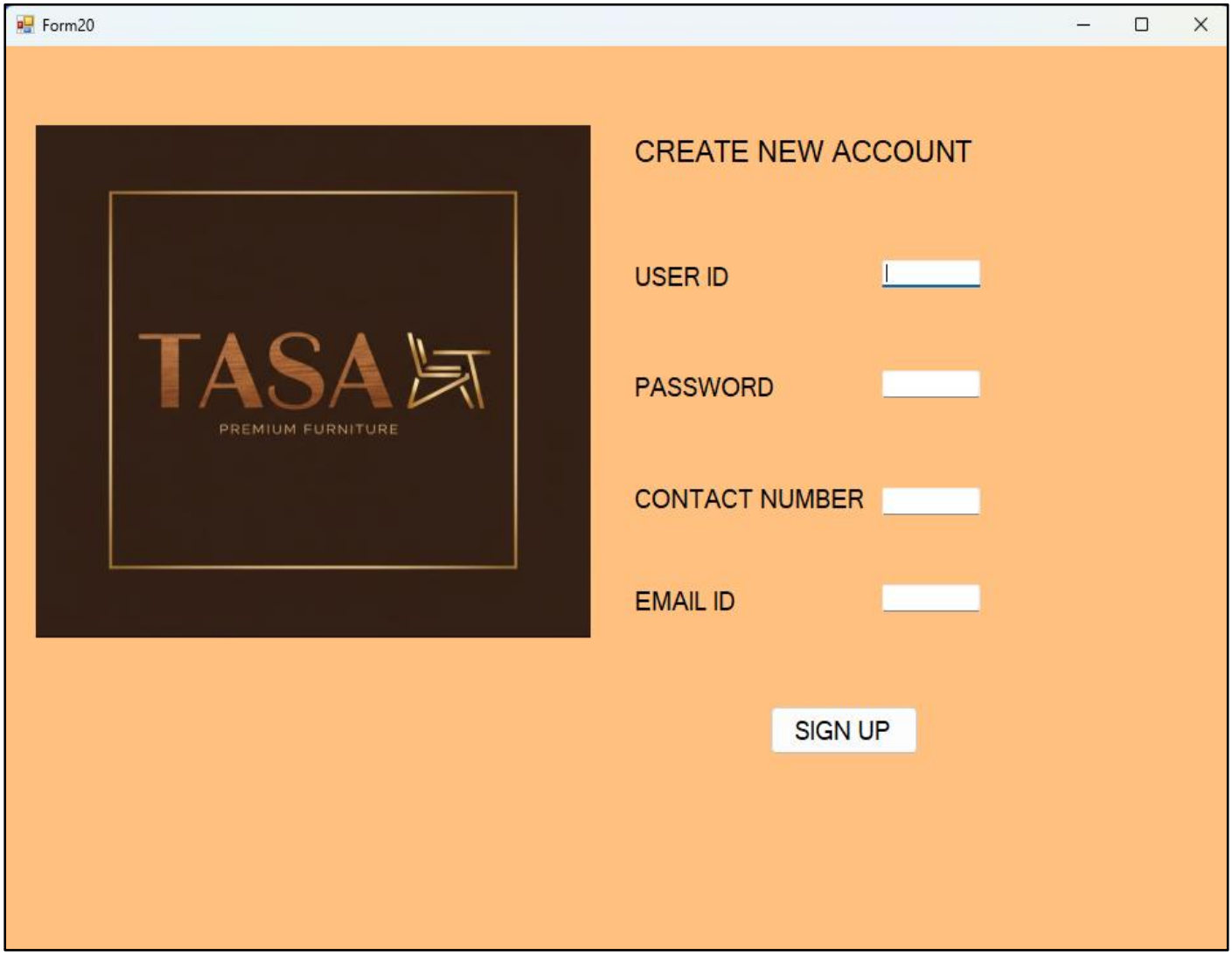


Code:

```
Imports System.Data.OleDb

Public Class Form19
    Dim con As New OleDb.OleDbConnection("Provider=Microsoft.Jet.OLEDB.4.0;Data
Source=Database2.mdb")
    Dim sql As String

    Dim cmd As New OleDb.OleDbCommand
    Dim dt As New DataTable
```

```
    Dim da As New OleDb.OleDbDataAdapter
    Dim i As Integer
    Private Sub LinkLabel1_LinkClicked(sender As Object, e As LinkLabelLinkClickedEventArgs) Handles
LinkLabel1.LinkClicked
        Form20.Show()
        Form20.Activate()
    End Sub
    Private Sub Button1_Click(sender As Object, e As EventArgs) Handles Button1.Click
        Try
            con.Open()
            Dim cmd As New OleDbCommand("SELECT * FROM Table2 WHERE [userid] = @user AND
[password] = @pass", con)
            cmd.Parameters.AddWithValue("@user", TextBox1.Text)
            cmd.Parameters.AddWithValue("@pass", TextBox2.Text)
            Dim reader As OleDbDataReader = cmd.ExecuteReader()
            If reader.Read() Then
                Dim user As String = reader("userid").ToString()
                Dim pass As String = reader("password").ToString()
                MessageBox.Show("Login Successfully!")
                Form1.Show()
                Form1.Activate()
            Else
                MsgBox("Invalid username and password!")
            End If
            reader.Close()
            con.Close()
        Catch ex As Exception
            MessageBox.Show("Error: " & ex.Message)
            con.Close()
        End Try
    End Sub
End Class
```

Register Window:



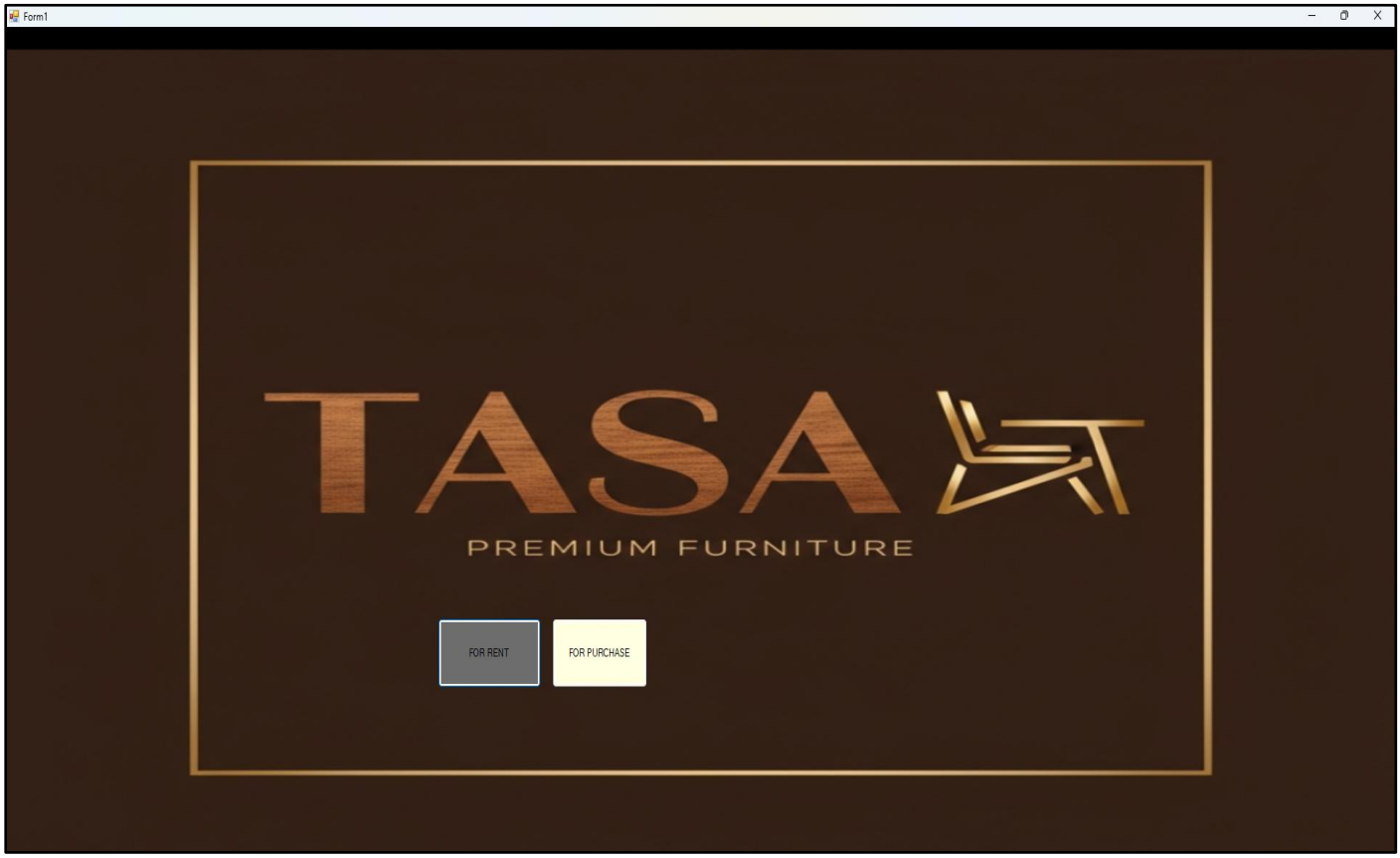
Code:

```
Imports System.Text.RegularExpressions

Public Class Form20
    Dim con As New OleDb.OleDbConnection("Provider=Microsoft.Jet.OLEDB.4.0;Data
Source=Database2.mdb")
    Dim sql As String
    Dim cmd As New OleDb.OleDbCommand
    Dim dt As New DataTable
    Dim da As New OleDb.OleDbDataAdapter
    Dim i As Integer
```

```
    Private Sub Button1_Click(sender As Object, e As EventArgs) Handles Button1.Click
        Try
            con.Open()
            sql = "INSERT INTO Table2([userid],[password],[contactnumber],[emailid]) values('" &
TextBox1.Text & "', '" & TextBox2.Text & "','" & TextBox3.Text & "','" & TextBox4.Text & "')"
            cmd.Connection = con
            cmd.CommandText = sql
            i = cmd.ExecuteNonQuery
            If i > 0 Then
                MsgBox("New User Signed Up successfully!")
            Else
                MsgBox("User Sign Up Unsuccessful....!")
            End If
        Catch ex As Exception
            MsgBox(ex.Message)
        Finally
            con.Close()
        End Try

        Me.Hide()
        ' Login.Show()
    End Sub
    Private Sub TextBox4_TextChanged(sender As Object, e As EventArgs) Handles TextBox4.TextChanged
        Dim emailPattern As String = "^[^@\s]+@[^@\s]+\.[^@\s]+$"
        Dim emailRegex As New Regex(emailPattern)
        If emailRegex.IsMatch(TextBox4.Text) Then
            TextBox4.BackColor = Color.LightGreen ' Valid email
        Else
            TextBox4.BackColor = Color.LightCoral ' Invalid email
        End If
    End Sub
    Private Sub TextBox3_TextChanged(sender As Object, e As EventArgs) Handles TextBox3.TextChanged
        Dim input As String = TextBox3.Text
        Dim filtered As String = ""
        For Each ch As Char In input
            If Char.IsDigit(ch) Then
```

```
            filtered &= ch
        End If
    Next
    If filtered <> input Then
        Dim pos As Integer = TextBox3.SelectionStart
        TextBox3.Text = filtered
        TextBox3.SelectionStart = Math.Min(pos, TextBox3.Text.Length)
    End If
  End Sub
End Class
```

Home Window:



Code:

```
Public Class Form1
    Private Sub Form1_Load(sender As Object, e As EventArgs) Handles MyBase.Load
        'PictureBox1.Image = Image.FromFile("C:\Users\USER\Downloads\WhatsApp Image 2025-12-17 at 15.52.48.jpeg")
    End Sub
    Private Sub Button1_Click_1(sender As Object, e As EventArgs) Handles Button1.Click
        Form2.Show()
        Form2.Activate()
    End Sub
    Private Sub Button2_Click(sender As Object, e As EventArgs) Handles Button2.Click
        Form16.Show()
        Form16.Activate()
    End Sub
End Class
```

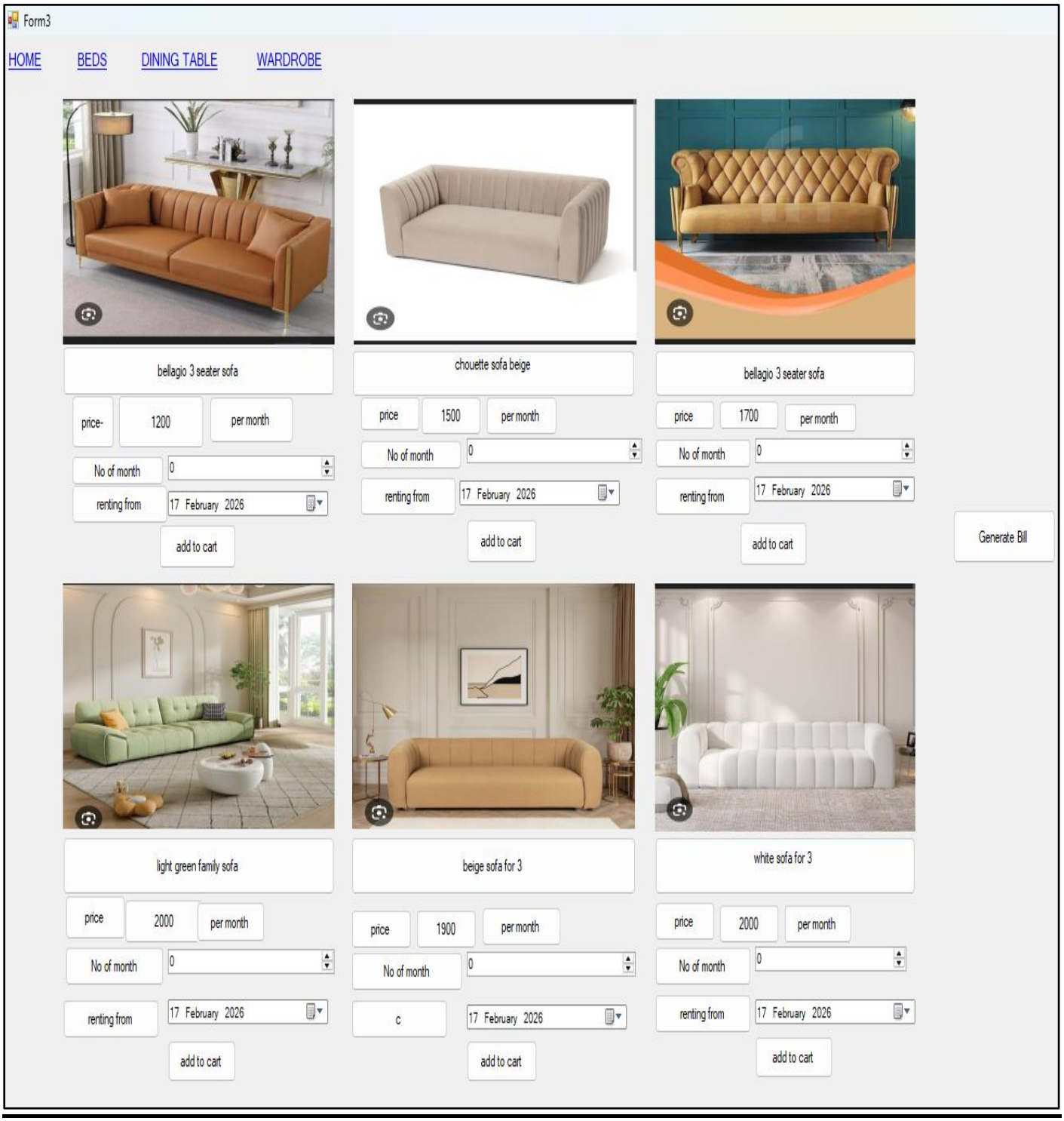
Rent Option Window:

Code:

```
Public Class Form16
    Private Sub PictureBox1_Click_1(sender As Object, e As EventArgs) Handles PictureBox1.Click
        Form7.Show()
        Form7.Activate()
    End Sub
    Private Sub PictureBox2_Click_1(sender As Object, e As EventArgs) Handles PictureBox2.Click
        Form8.Show()
        Form8.Activate()
```

```
    End Sub
    Private Sub PictureBox3_Click_1(sender As Object, e As EventArgs) Handles PictureBox3.Click
        Form9.Show()
        Form9.Activate()
    End Sub
    Private Sub PictureBox4_Click_1(sender As Object, e As EventArgs) Handles PictureBox4.Click
        Form10.Show()
        Form10.Activate()
    End Sub
End Class
```

Sofa Window:



Code:

```
Public Class Form3
    Dim con As New OleDb.OleDbConnection("Provider=Microsoft.Jet.OLEDB.4.0;Data
Source=Database2.mdb")
```

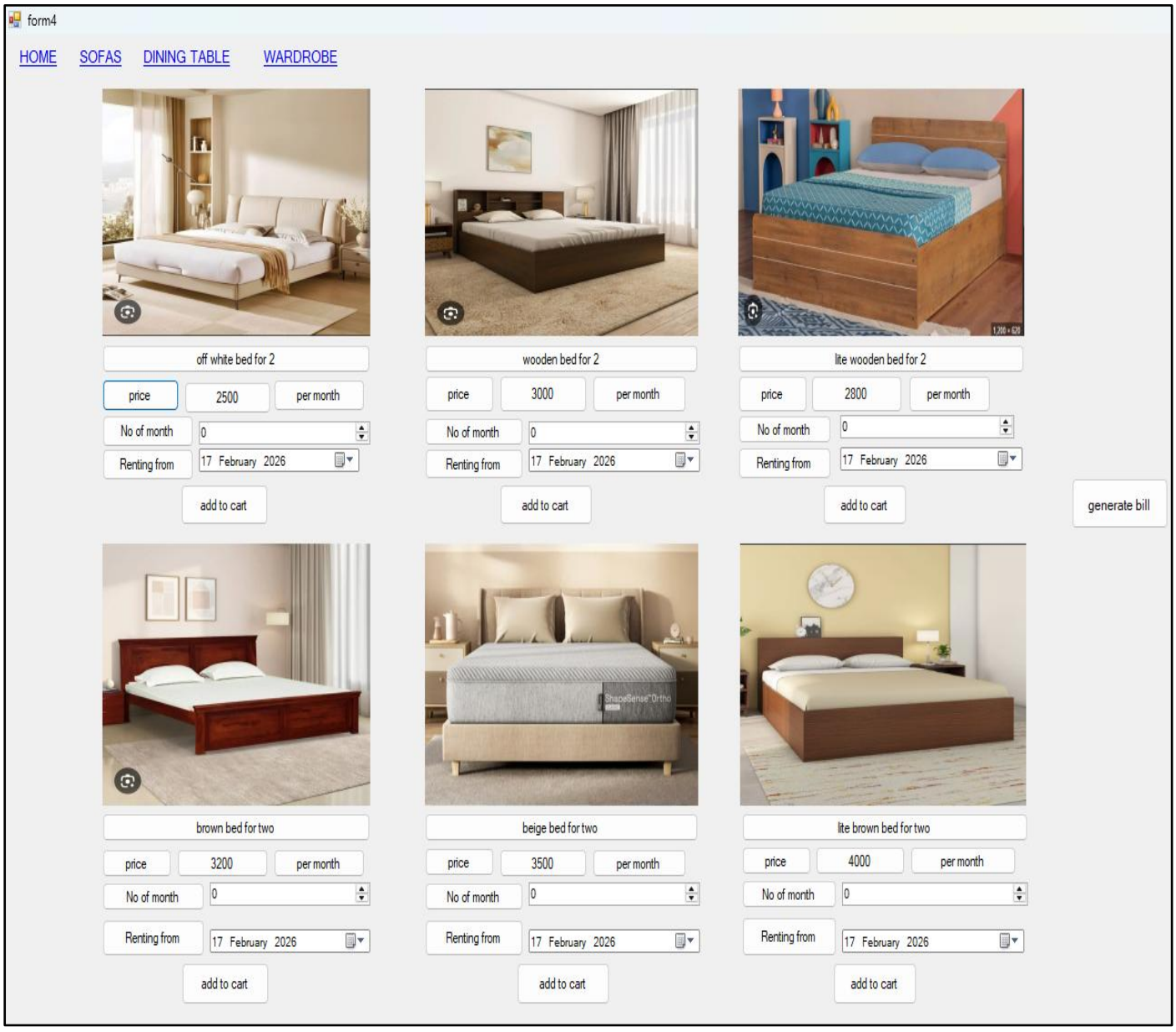
```
Public sql As String
Public mystring As String
Dim cmd As New OleDb.OleDbCommand
Dim dt As New DataTable
Dim da As New OleDb.OleDbDataAdapter
Dim i As Integer
Dim record, update As String
Dim read, existing As String
Dim stock, iname, NOM, RENTED, price, No As Integer
Dim total As Integer = 0

Private Sub Button11_Click(sender As Object, e As EventArgs) Handles Button11.Click
    Name = Button5.Text
    NOM = NumericUpDown1.Value
    price = Button27.Text
    NOM = NumericUpDown5.Value

    price = price * NOM
    Form11.ITEMS.Items.Add(Name)
    Form11.MONTHS_RENTED_FOR.Items.Add(NOM)
    Form11.PRICE.Items.Add(price)
    total = total + price
    Form11.Button9.Text = total

    MsgBox("Item added Successfully")
End Sub

Public Sub Button7_Click(sender As Object, e As EventArgs) Handles Button7.Click
    Name = Button1.Text
    NOM = NumericUpDown1.Value
    price = Button15.Text
    NOM = NumericUpDown1.Value

    price = price * NOM
    Form11.ITEMS.Items.Add(Name)
    Form11.MONTHS_RENTED_FOR.Items.Add(NOM)
```

```
    Form11.PRICE.Items.Add(price)
    total = total + price
    Form11.Button9.Text = total

    MsgBox("Item added Successfully")

End Sub
Private Sub Button14_Click(sender As Object, e As EventArgs) Handles Button14.Click
    Form11.Show()
    Form11.Activate()

End Sub

Private Sub Button8_Click(sender As Object, e As EventArgs) Handles Button8.Click
    Name = Button2.Text
    NOM = NumericUpDown1.Value
    price = Button18.Text
    NOM = NumericUpDown2.Value

    price = price * NOM
    Form11.ITEMS.Items.Add(Name)
    Form11.MONTHS_RENTED_FOR.Items.Add(NOM)
    Form11.PRICE.Items.Add(price)
    total = total + price
    Form11.Button9.Text = total

    MsgBox("Item added Successfully")

End Sub
Private Sub Button9_Click(sender As Object, e As EventArgs) Handles Button9.Click
    Name = Button3.Text
    NOM = NumericUpDown1.Value
    price = Button21.Text
    NOM = NumericUpDown3.Value

    price = price * NOM
```

```
    Form11.ITEMS.Items.Add(Name)
    Form11.MONTHS_RENTED_FOR.Items.Add(NOM)
    Form11.PRICE.Items.Add(price)
    total = total + price
    Form11.Button9.Text = total

    MsgBox("Item added Successfully")
End Sub

Private Sub Button10_Click(sender As Object, e As EventArgs) Handles Button10.Click
    Name = Button4.Text
    NOM = NumericUpDown1.Value
    price = Button24.Text
    NOM = NumericUpDown4.Value

    price = price * NOM
    Form11.ITEMS.Items.Add(Name)
    Form11.MONTHS_RENTED_FOR.Items.Add(NOM)
    Form11.PRICE.Items.Add(price)
    total = total + price
    Form11.Button9.Text = total
    MsgBox("Item added Successfully")
End Sub

Private Sub Button13_Click(sender As Object, e As EventArgs) Handles Button13.Click
    Name = Button6.Text
    NOM = NumericUpDown1.Value
    price = Button30.Text
    NOM = NumericUpDown6.Value

    price = price * NOM
    Form11.ITEMS.Items.Add(Name)
    Form11.MONTHS_RENTED_FOR.Items.Add(NOM)
    Form11.PRICE.Items.Add(price)
    total = total + price
    Form11.Button9.Text = total
```

```
        MsgBox("Item added Successfully")
    End Sub

    Private Sub LinkLabel1_LinkClicked(sender As Object, e As LinkLabelLinkClickedEventArgs) Handles
LinkLabel1.LinkClicked
        Form1.Show()
        Form1.Activate()
    End Sub

    Private Sub LinkLabel2_LinkClicked(sender As Object, e As LinkLabelLinkClickedEventArgs) Handles
LinkLabel2.LinkClicked
        Form4.Show()
        Form4.Activate()
    End Sub

    Private Sub LinkLabel3_LinkClicked(sender As Object, e As LinkLabelLinkClickedEventArgs) Handles
LinkLabel3.LinkClicked
        Form5.Show()
        Form5.Activate()
    End Sub


    Private Sub LinkLabel4_LinkClicked(sender As Object, e As LinkLabelLinkClickedEventArgs) Handles
LinkLabel4.LinkClicked
        Form6.Show()
        Form6.Activate()
    End Sub
End Class
```

Beds Window:



Code:

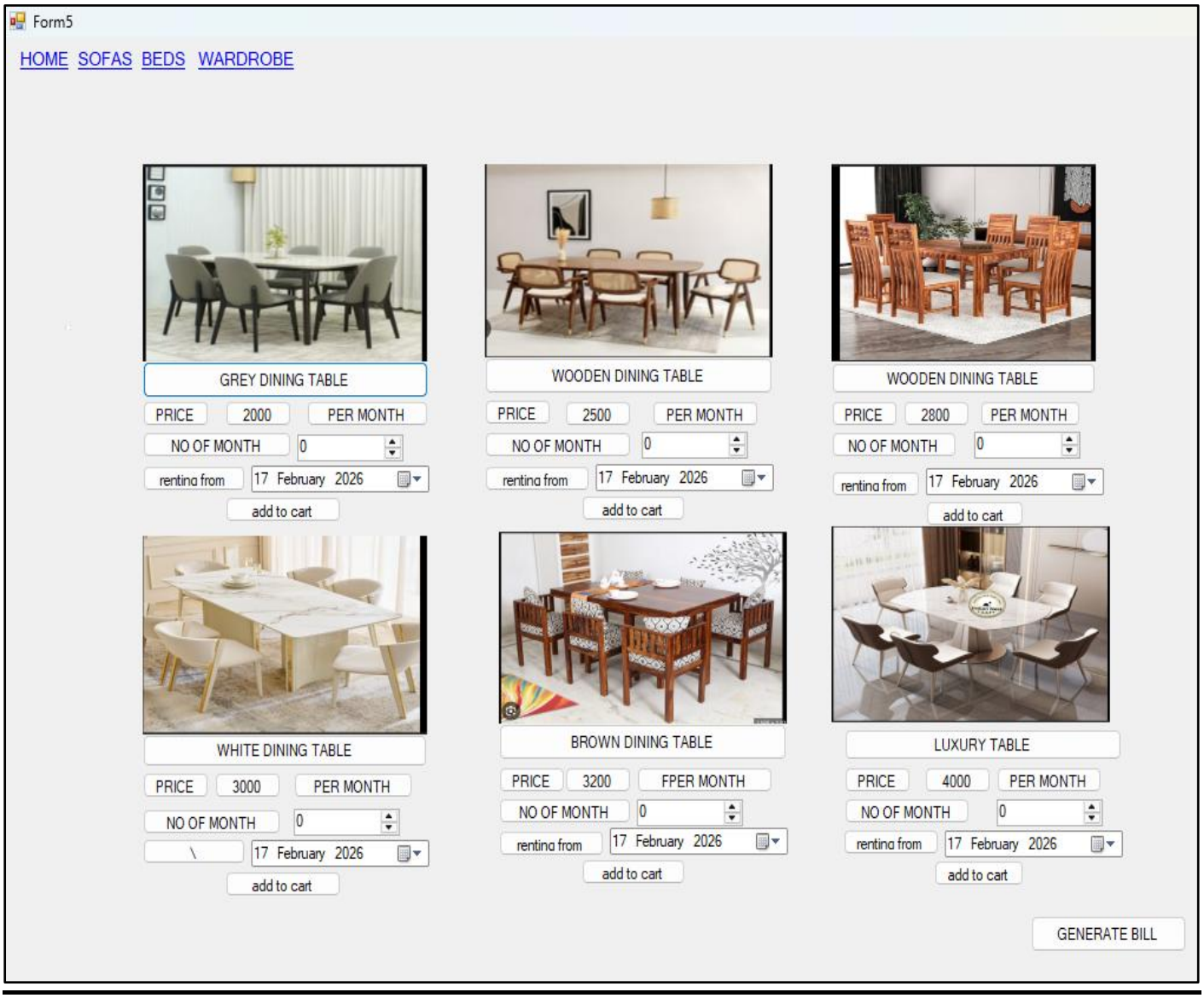
```
Public Class Form4
    Dim iname, NOM, RENTED, price, No As Integer
    Dim total As Integer = 0
    Private Sub Button3_Click(sender As Object, e As EventArgs) Handles Button3.Click
        Name = Button19.Text
        NOM = NumericUpDown1.Value
        price = Button2.Text
```

```
    NOM = NumericUpDown1.Value
    price = price * NOM
    Form11.ITEMS.Items.Add(Name)
    Form11.MONTHS_RENTED_FOR.Items.Add(NOM)
    Form11.PRICE.Items.Add(price)
    total = total + price
    Form11.Button9.Text = total
    MsgBox("Items added successfully")
End Sub
Private Sub Button43_Click(sender As Object, e As EventArgs) Handles Button43.Click
    Form11.Show()
    Form11.Activate()
End Sub
Private Sub Button6_Click(sender As Object, e As EventArgs) Handles Button6.Click
    Name = Button20.Text
    NOM = NumericUpDown1.Value
    price = Button5.Text
    NOM = NumericUpDown2.Value

    price = price * NOM
    Form11.ITEMS.Items.Add(Name)
    Form11.MONTHS_RENTED_FOR.Items.Add(NOM)
    Form11.PRICE.Items.Add(price)
    total = total + price
    Form11.Button9.Text = total
    MsgBox("Items added successfully")
End Sub
Private Sub Button9_Click(sender As Object, e As EventArgs) Handles Button9.Click
    Name = Button21.Text
    NOM = NumericUpDown1.Value
    price = Button8.Text
    NOM = NumericUpDown3.Value

    price = price * NOM
    Form11.ITEMS.Items.Add(Name)
    Form11.MONTHS_RENTED_FOR.Items.Add(NOM)
```

```
    Form11.PRICE.Items.Add(price)
    total = total + price
    Form11.Button9.Text = total
    MsgBox("Items added successfully")
End Sub
Private Sub Button16_Click(sender As Object, e As EventArgs) Handles Button16.Click
    Name = Button22.Text
    NOM = NumericUpDown1.Value
    price = Button11.Text
    NOM = NumericUpDown4.Value

    price = price * NOM
    Form11.ITEMS.Items.Add(Name)
    Form11.MONTHS_RENTED_FOR.Items.Add(NOM)
    Form11.PRICE.Items.Add(price)
    total = total + price
    Form11.Button9.Text = total
    MsgBox("Items added successfully")
End Sub
Private Sub Button17_Click(sender As Object, e As EventArgs) Handles Button17.Click
    Name = Button23.Text
    NOM = NumericUpDown1.Value
    price = Button13.Text
    NOM = NumericUpDown5.Value
    price = price * NOM
    Form11.ITEMS.Items.Add(Name)
    Form11.MONTHS_RENTED_FOR.Items.Add(NOM)
    Form11.PRICE.Items.Add(price)
    total = total + price
    Form11.Button9.Text = total
    MsgBox("Items added successfully")
End Sub
Private Sub Button18_Click(sender As Object, e As EventArgs) Handles Button18.Click
    Name = Button24.Text
    NOM = NumericUpDown1.Value
    price = Button15.Text
```

```
        NOM = NumericUpDown6.Value
        price = price * NOM
        Form11.ITEMS.Items.Add(Name)
        Form11.MONTHS_RENTED_FOR.Items.Add(NOM)
        Form11.PRICE.Items.Add(price)
        total = total + price
        Form11.Button9.Text = total
        MsgBox("Items added successfully")
    End Sub
    Private Sub LinkLabel2_LinkClicked(sender As Object, e As LinkLabelLinkClickedEventArgs) Handles
LinkLabel2.LinkClicked
        Form3.Show()
        Form3.Activate()
    End Sub
    Private Sub LinkLabel4_LinkClicked(sender As Object, e As LinkLabelLinkClickedEventArgs) Handles
LinkLabel4.LinkClicked
        Form6.Show()
        Form6.Activate()
    End Sub
    Private Sub LinkLabel3_LinkClicked(sender As Object, e As LinkLabelLinkClickedEventArgs) Handles
LinkLabel3.LinkClicked
        Form5.Show()
        Form5.Activate()
    End Sub
    Private Sub Button44_Click(sender As Object, e As EventArgs) Handles Button44.Click
        Form11.Show()
        Form11.Activate()
    End Sub
End Class
```

Dining Table Window:



Code:

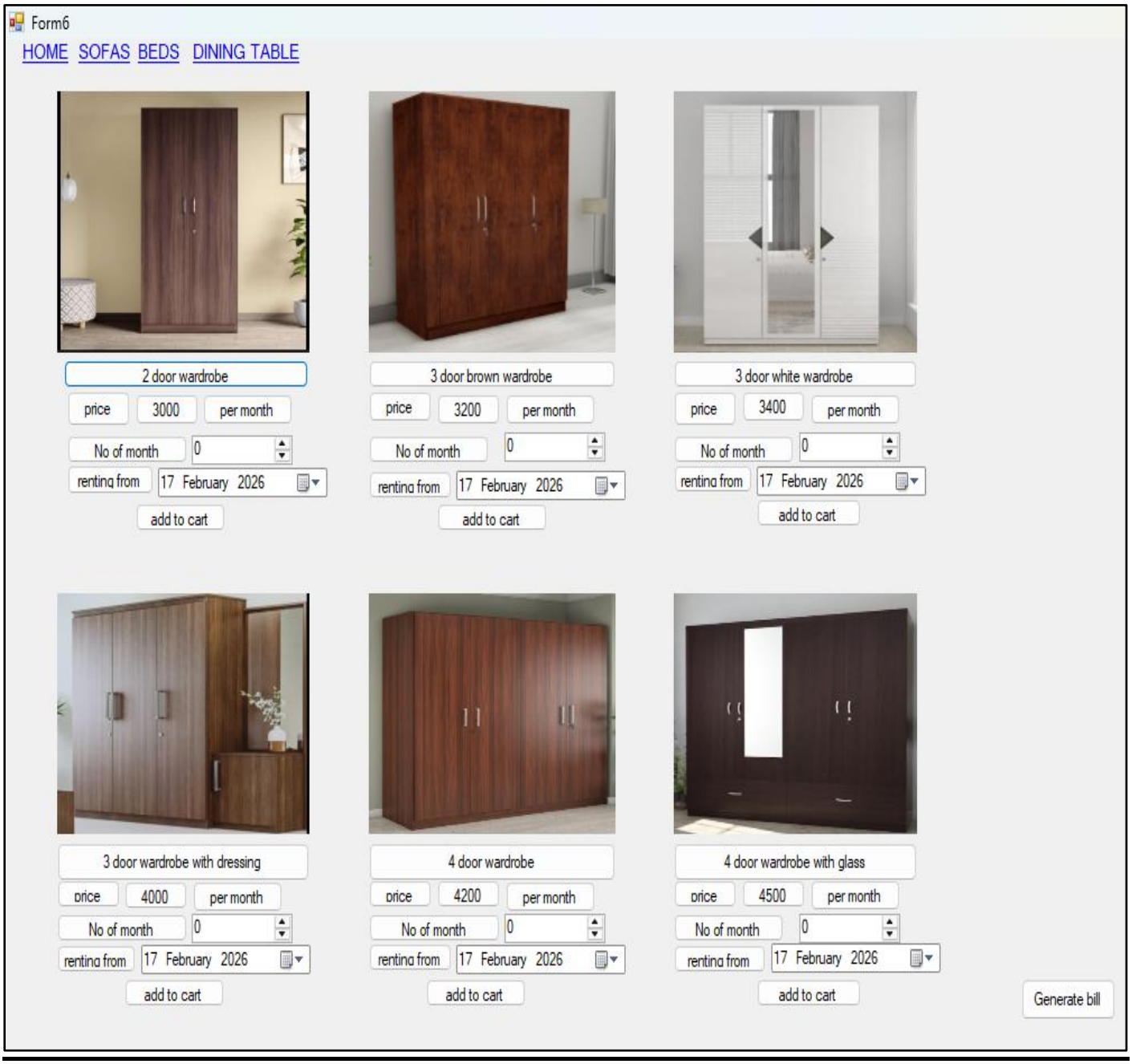
```
Public Class Form5
    Dim iname, NOM, price, total As Integer
    Private Sub Button7_Click(sender As Object, e As EventArgs) Handles Button7.Click
        Name = Button2.Text
        NOM = NumericUpDown1.Value
        price = Button35.Text
        NOM = NumericUpDown2.Value()
        price = price * NOM
```

```
    Form11.ITEMS.Items.Add(Name)
    Form11.MONTHS_RENTED_FOR.Items.Add(NOM)
    Form11.PRICE.Items.Add(price)
    total = total + price
    Form11.Button9.Text = total
    MsgBox("Items added successfully")
End Sub
Private Sub Button33_Click(sender As Object, e As EventArgs) Handles Button33.Click
    Name = Button1.Text
    NOM = NumericUpDown1.Value
    price = Button4.Text
    NOM = NumericUpDown1.Value()
    price = price * NOM
    Form11.ITEMS.Items.Add(Name)
    Form11.MONTHS_RENTED_FOR.Items.Add(NOM)
    Form11.PRICE.Items.Add(price)
    total = total + price
    Form11.Button9.Text = total
    MsgBox("Items added successfully")
End Sub
Private Sub Button44_Click(sender As Object, e As EventArgs) Handles Button44.Click
    Form11.Show()
    Form11.Activate()
End Sub
Private Sub Button12_Click(sender As Object, e As EventArgs) Handles Button12.Click
    Name = Button13.Text
    NOM = NumericUpDown1.Value
    price = Button15.Text
    NOM = NumericUpDown3.Value()
    price = price * NOM
    Form11.ITEMS.Items.Add(Name)
    Form11.MONTHS_RENTED_FOR.Items.Add(NOM)
    Form11.PRICE.Items.Add(price)
    total = total + price
    Form11.Button9.Text = total
    MsgBox("Items added successfully")
```

```
End Sub
Private Sub Button20_Click(sender As Object, e As EventArgs) Handles Button20.Click
    Name = Button16.Text
    NOM = NumericUpDown1.Value
    price = Button17.Text
    NOM = NumericUpDown4.Value()
    price = price * NOM
    Form11.ITEMS.Items.Add(Name)
    Form11.MONTHS_RENTED_FOR.Items.Add(NOM)
    Form11.PRICE.Items.Add(price)
    total = total + price
    Form11.Button9.Text = total
    MsgBox("Items added successfully")
End Sub
Private Sub Button41_Click(sender As Object, e As EventArgs) Handles Button41.Click
    Name = Button21.Text
    NOM = NumericUpDown1.Value
    price = Button22.Text
    NOM = NumericUpDown5.Value()
    price = price * NOM
    Form11.ITEMS.Items.Add(Name)
    Form11.MONTHS_RENTED_FOR.Items.Add(NOM)
    Form11.PRICE.Items.Add(price)
    total = total + price
    Form11.Button9.Text = total
    MsgBox("Items added successfully")
End Sub
Private Sub Button29_Click(sender As Object, e As EventArgs) Handles Button29.Click
    Name = Button26.Text
    NOM = NumericUpDown1.Value
    price = Button28.Text
    NOM = NumericUpDown7.Value()
    price = price * NOM
    Form11.ITEMS.Items.Add(Name)
    Form11.MONTHS_RENTED_FOR.Items.Add(NOM)
    Form11.PRICE.Items.Add(price)
```

```
        total = total + price
        Form11.Button9.Text = total
        MsgBox("Items added successfully")
    End Sub
    Private Sub LinkLabel4_LinkClicked(sender As Object, e As LinkLabelLinkClickedEventArgs) Handles
LinkLabel4.LinkClicked
        Form6.Show()
        Form6.Activate()
    End Sub
    Private Sub LinkLabel3_LinkClicked(sender As Object, e As LinkLabelLinkClickedEventArgs) Handles
LinkLabel3.LinkClicked
        Form4.Show()
        Form4.Activate()
    End Sub
    Private Sub LinkLabel2_LinkClicked(sender As Object, e As LinkLabelLinkClickedEventArgs) Handles
LinkLabel2.LinkClicked
        Form3.Show()
        Form3.Activate()
    End Sub
    Private Sub LinkLabel1_LinkClicked(sender As Object, e As LinkLabelLinkClickedEventArgs) Handles
LinkLabel1.LinkClicked
        Form1.Show()
        Form1.Activate()
    End Sub
End Class
```

Wardrobe Window:



Code;

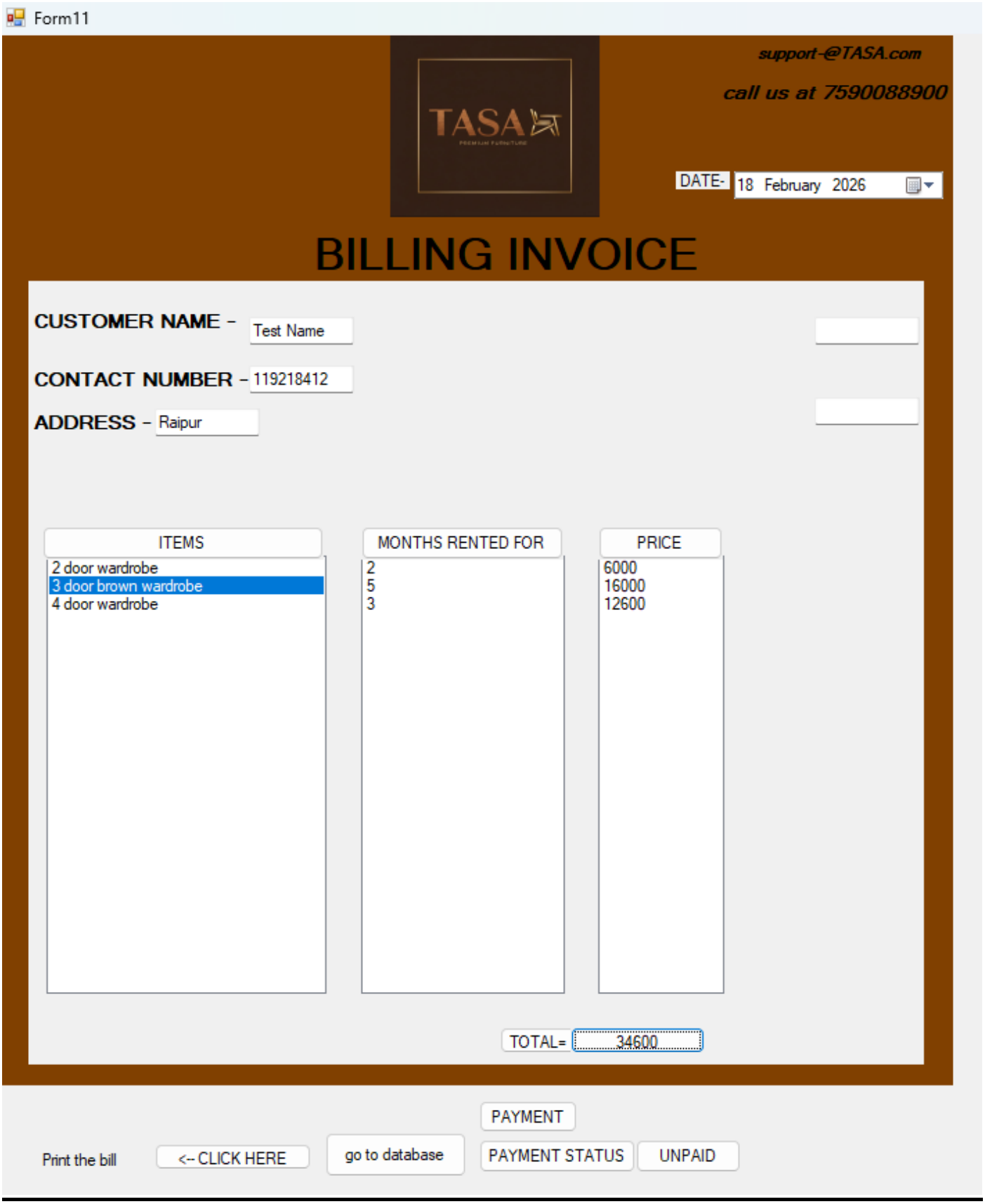
```
Public Class Form6
    Dim iname, NOM, RENTED, price, No As Integer
    Dim total As Integer = 0
    Private Sub Button28_Click(sender As Object, e As EventArgs) Handles Button28.Click
        Name = Button23.Text
```

```
    NOM = NumericUpDown1.Value
    price = Button29.Text
    NOM = NumericUpDown2.Value
    price = price * NOM
    Form11.ITEMS.Items.Add(Name)
    Form11.MONTHS_RENTED_FOR.Items.Add(NOM)
    Form11.PRICE.Items.Add(price)
    total = total + price
    Form11.Button9.Text = total
    MsgBox("Items added successfully")
End Sub
Private Sub Button6_Click(sender As Object, e As EventArgs) Handles Button6.Click
    Name = Button1.Text
    NOM = NumericUpDown1.Value
    price = Button3.Text
    NOM = NumericUpDown1.Value
    price = price * NOM
    Form11.ITEMS.Items.Add(Name)
    Form11.MONTHS_RENTED_FOR.Items.Add(NOM)
    Form11.PRICE.Items.Add(price)
    total = total + price
    Form11.Button9.Text = total
    MsgBox("Items added successfully")
End Sub
Private Sub Button13_Click(sender As Object, e As EventArgs) Handles Button13.Click
    Name = Button8.Text
    NOM = NumericUpDown1.Value
    price = Button10.Text
    NOM = NumericUpDown2.Value
    price = price * NOM
    Form11.ITEMS.Items.Add(Name)
    Form11.MONTHS_RENTED_FOR.Items.Add(NOM)
    Form11.PRICE.Items.Add(price)
    total = total + price
    Form11.Button9.Text = total
    MsgBox("Items added successfully")
```

```
End Sub
Private Sub Button21_Click(sender As Object, e As EventArgs) Handles Button21.Click
    Name = Button16.Text
    NOM = NumericUpDown1.Value
    price = Button18.Text
    NOM = NumericUpDown4.Value
    price = price * NOM
    Form11.ITEMS.Items.Add(Name)
    Form11.MONTHS_RENTED_FOR.Items.Add(NOM)
    Form11.PRICE.Items.Add(price)
    total = total + price
    Form11.Button9.Text = total
    MsgBox("Items added successfully")
End Sub
Private Sub Button36_Click(sender As Object, e As EventArgs) Handles Button36.Click
    Name = Button30.Text
    NOM = NumericUpDown1.Value
    price = Button32.Text
    NOM = NumericUpDown5.Value
    price = price * NOM
    Form11.ITEMS.Items.Add(Name)
    Form11.MONTHS_RENTED_FOR.Items.Add(NOM)
    Form11.PRICE.Items.Add(price)
    total = total + price
    Form11.Button9.Text = total
    MsgBox("Items added successfully")
End Sub
Private Sub Button41_Click(sender As Object, e As EventArgs) Handles Button41.Click
    Name = Button37.Text
    NOM = NumericUpDown1.Value
    price = Button39.Text
    NOM = NumericUpDown2.Value
    price = price * NOM
    Form11.ITEMS.Items.Add(Name)
    Form11.MONTHS_RENTED_FOR.Items.Add(NOM)
    Form11.PRICE.Items.Add(price)
```

```
        total = total + price
        Form11.Button9.Text = total
        MsgBox("Items added successfully")
    End Sub
    Private Sub Button15_Click(sender As Object, e As EventArgs) Handles Button15.Click
        Form11.Show()
        Form11.Activate()
    End Sub
    Private Sub LinkLabel4_LinkClicked(sender As Object, e As LinkLabelLinkClickedEventArgs) Handles
LinkLabel4.LinkClicked
        Form5.Show()
        Form5.Activate()
    End Sub
    Private Sub LinkLabel3_LinkClicked(sender As Object, e As LinkLabelLinkClickedEventArgs) Handles
LinkLabel3.LinkClicked
        Form4.Show()
        Form4.Activate()
    End Sub
    Private Sub LinkLabel2_LinkClicked(sender As Object, e As LinkLabelLinkClickedEventArgs) Handles
LinkLabel2.LinkClicked
        Form3.Show()
        Form3.Activate()
    End Sub
End Class
```

Bill Invoice Window:



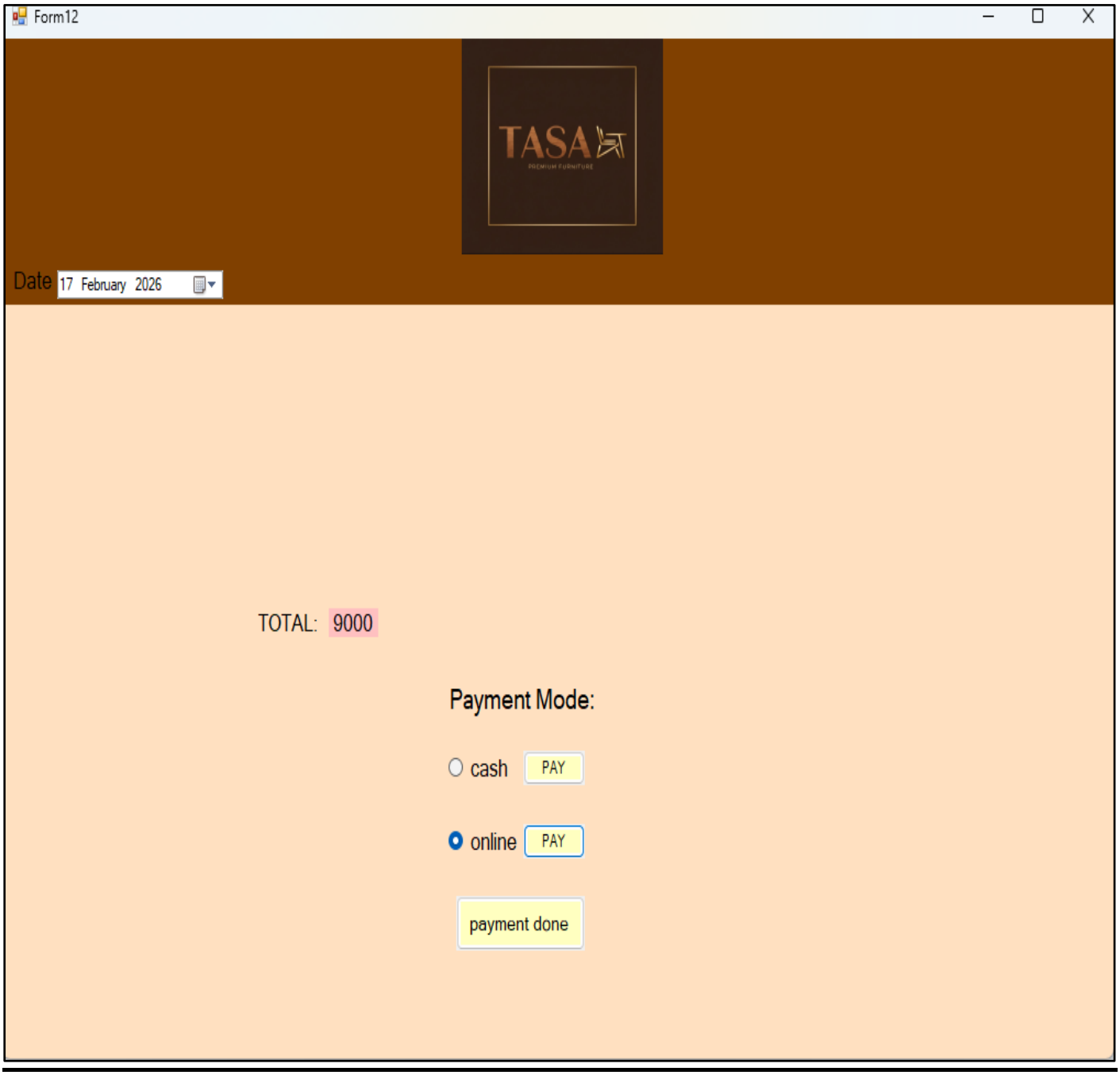
Code:

```
Imports System.Drawing
Imports System.Drawing.Printing
Imports System.Data.OleDb

Public Class Form11
    Dim con As New OleDb.OleDbConnection("Provider=Microsoft.Jet.OLEDB.4.0;Data
Source=Database2.mdb")
    Dim sql As String
    Dim cmd As New OleDb.OleDbCommand
    Dim dt As New DataTable
    Dim da As New OleDb.OleDbDataAdapter
    Dim i As Integer
    Private Sub Form11_Load(sender As Object, e As EventArgs) Handles MyBase.Load
        PrintPreviewDialog1.Document = PrintDocument1
    End Sub
    Private Sub PrintDocument1_PrintPage(sender As Object, e As PrintPageEventArgs) Handles
PrintDocument1.PrintPage
        Using bmp As New Bitmap(Me.ClientSize.Width, Me.ClientSize.Height)
            Me.DrawToBitmap(bmp, New Rectangle(0, 0, bmp.Width, bmp.Height))
            Dim marginBounds As Rectangle = e.MarginBounds
            Dim scale As Single = Math.Min(marginBounds.Width / bmp.Width, marginBounds.Height /
bmp.Height)
            Dim drawWidth As Integer = CInt(bmp.Width * scale)
            Dim drawHeight As Integer = CInt(bmp.Height * scale)
            Dim drawX As Integer = marginBounds.Left + CInt((marginBounds.Width - drawWidth) / 2)
            Dim drawY As Integer = marginBounds.Top + CInt((marginBounds.Height - drawHeight) / 2)
            e.Graphics.DrawImage(bmp, drawX, drawY, drawWidth, drawHeight)
        End Using
        e.HasMorePages = False
    End Sub
    Private Sub Label2_Click(sender As Object, e As EventArgs) Handles Label2.Click
        Dim wasButton1Visible = Button1.Visible
        Dim wasButton2Visible = Button2.Visible
```

```
        Dim wasButton6Visible = Button6.Visible
        Dim wasButton7Visible = Button7.Visible
        Dim wasLabel2Visible = Label2.Visible
        Button1.Visible = False
        Button2.Visible = False
        Button6.Visible = False
        Button7.Visible = False
        Label2.Visible = False
        Try
            PrintPreviewDialog1.Document = PrintDocument1
            PrintPreviewDialog1.ShowDialog()
            con.Open()
            sql = "INSERT INTO Table1 ([date],cname,customermobile,noofitem,totalbill,paymentmode) values
(?, ?, ?, ?, ?, ?)"
            cmd.Connection = con
            cmd.CommandText = sql
            cmd.Parameters.Clear()
            cmd.Parameters.AddWithValue("?", DateTimePicker3.Value)
            cmd.Parameters.AddWithValue("?", TextBox1.Text)
            cmd.Parameters.AddWithValue("?", TextBox2.Text)
            cmd.Parameters.AddWithValue("?", TextBox4.Text)
            cmd.Parameters.AddWithValue("?", Button9.Text)
            cmd.Parameters.AddWithValue("?", TextBox5.Text)
            i = cmd.ExecuteNonQuery()
            If i > 0 Then
                MsgBox("New record has been inserted successfully!")
            Else
                MsgBox("No record has been inserted successfully!")
            End If
        Catch ex As Exception
            MsgBox(ex.Message)
        Finally
            con.Close()
            Button1.Visible = wasButton1Visible
            Button2.Visible = wasButton2Visible
            Button6.Visible = wasButton6Visible
```

```
            Button7.Visible = wasButton7Visible
            Label2.Visible = wasLabel2Visible
        End Try
    End Sub
    Private Sub Button2_Click_1(sender As Object, e As EventArgs) Handles Button2.Click
        Form12.Label5.Text = Button9.Text
        Form12.Show()
        Form12.Activate()
    End Sub
    Private Sub Button8_Click(sender As Object, e As EventArgs) Handles Button8.Click
        Form18.Show()
        Form18.Activate()
    End Sub
End Class
```

Payment Window:



Code:

```
Public Class Form12
    Private Sub Button2_Click(sender As Object, e As EventArgs) Handles Button2.Click
        Form13.Show()
        Form13.Activate()
    End Sub
```

```
    Private Sub Button1_Click_1(sender As Object, e As EventArgs)
        Form11.Button6.Text = "Paid"
        Form11.Show()
        Form11.Activate()
    End Sub
    Private Sub Button1_Click_2(sender As Object, e As EventArgs)
        Form11.Button6.Text = "Paid"
        Form11.Show()
        Form11.Activate()
    End Sub
    Private Sub Button4_Click_1(sender As Object, e As EventArgs) Handles Button4.Click
        Form11.Button6.Text = "Paid"
        Form11.Show()
        Form11.Activate()
    End Sub
End Class
```

Online Payment Window:

Code:

```
Public Class Form13
    Private Sub Button2_Click(sender As Object, e As EventArgs) Handles Button2.Click
        Form11.Button7.Text = "Paid"
        Form11.Show()
        Form11.Activate()
        Me.Hide()
    End Sub
    Private Sub Button3_Click(sender As Object, e As EventArgs)
        Form14.Button7.Text = "Paid"
        Form14.Show()
        Form14.Activate()
        Me.Hide()
    End Sub
End Class
```

12. LIMITATIONS

1. **Platform Dependency (Windows Only):** The current **Furniture Rental Management System** is built using the .NET Framework, which limits its operation primarily to computers running the Microsoft Windows operating system. This restricts accessibility for users with macOS or Linux environments and prevents the shop owner from accessing business data via mobile devices or tablets while away from the shop.
2. **Lack of Online Customer Access:** As a desktop-based application, the system operates offline within the shop's local network. Consequently, customers cannot browse the furniture catalog, check stock availability, or place booking requests remotely from their homes. All transactions require the physical presence of the customer or manual entry by a staff member via phone orders.
3. **Database Scalability Constraints:** The system utilizes **Microsoft Access** as its backend database. While efficient for small to medium-sized operations, Access has performance limitations regarding file size (2GB limit) and concurrent user access. As the business grows and transaction records accumulate over years, the database may experience slower query response times compared to enterprise-level solutions like SQL Server.

13. FUTURE SCOPE OF THE PROJECT

1. **Web Portal Integration:** The project can be expanded by developing a companion website or web portal. This would allow customers to browse the furniture inventory online, view pricing, and submit "Booking Requests" that sync with the shop's desktop application, bridging the gap between offline operations and online convenience.
2. **Barcode/QR Code Integration:** Implementing a Barcode or QR Code scanning feature would significantly speed up the inventory management process. By tagging each furniture piece with a unique code, staff could simply scan items during "Issue" and "Return" processes to instantly update stock levels and verify item conditions, eliminating manual data entry errors.
3. **Migration to Cloud Database (SQL Server):** To address scalability limitations, the backend can be migrated from MS Access to a cloud-based **SQL Server** or **Azure Database**. This would enable multi-branch support, allowing a chain of rental shops to share a centralized inventory and customer database in real-time.
4. **Automated SMS and Email Reminders:** The system can be enhanced to integrate with SMS gateways (APIs). This would allow the application to automatically send payment reminders to customers three days before their rental period expires, as well as confirmation messages when furniture is successfully returned.
5. **Mobile App for Delivery Staff:** A complementary mobile application could be developed for delivery personnel. This would allow them to capture digital signatures upon delivery, upload photos of the furniture's condition at the customer's site, and update the order status to "Delivered" instantly from the field.

14. CONCLUSION

The **Furniture Rental Management System** project successfully addresses the key objectives of providing a stable and efficient platform for managing the daily operations of a furniture rental business. By replacing error-prone manual ledgers with a digital inventory and billing system, the platform enhances operational accuracy. For administrators, the system simplifies the complex tasks of tracking stock availability across categories (e.g., Living Room, Office), calculating rental durations, and managing customer identity proofs.

The scope of the project includes creating a robust desktop application capable of handling the entire rental lifecycle—from issuing items to processing returns—with a focus on data security and user-friendly interface design. The **VB.NET (Windows Forms)** architecture ensures that the system is responsive and easy to navigate for staff with varying technical skills, while the **Microsoft Access** database provides a reliable and portable storage solution for critical business records.

In conclusion, the **Furniture Rental Management System** provides a practical, cost-effective, and secure solution for modernizing rental shops. It meets the platform’s goals by ensuring accurate billing, reducing "ghost inventory" issues, and streamlining workflow. With future enhancements such as Barcode integration and cloud connectivity, the system is poised to scale alongside the business's growth.

15. BIBLIOGRAPHY

The Bibliography contains references to all the documents and resources that were referred to for the creation and successful completion of the **Furniture Rental Management System**. It contains the names of the referred software engineering documents, VB.NET documentation, and database standards.

- **Book:** *Visual Basic 2012 How to Program* by Paul Deitel and Harvey Deitel.
- **Book:** *Database Systems: Design, Implementation, & Management* by Carlos Coronel (Section on MS Access).
- **Documentation:** Microsoft Learn - .NET Documentation (https://learn.microsoft.com/en-us/dotnet/).
- **Documentation:** Windows Forms (WinForms) Guide (https://learn.microsoft.com/en-us/dotnet/desktop/winforms/).
- **Resource:** https://www.connectionstrings.com/ (Reference for OLEDB Connection Strings).
- **Resource:** https://stackoverflow.com (Community support for VB.NET error handling).
- **Tool:** Microsoft Visual Studio 2019 (Integrated Development Environment).
- **Tool:** Microsoft Access (Database Design Tool).
- **Tutorials:** "ProgrammingKnowledge - VB.NET & Access Database Tutorial" on YouTube.
- **Tutorials:** "FoxLearn - Furniture Store Management System in VB.NET" on YouTube.